

Variations on Linear PCF (Extended Abstract)

Valeria de Paiva and Eike Ritter*

School of Computer Science, University of Birmingham

Abstract. This paper shows that the Girard translation between the simply typed λ -calculus and the linear λ -calculus preserves and reflects observational equality. This holds not only for the call-by-name calculi but also for the call-by-value calculi. The proof consists of adapting the well-known techniques of logical relations to a linear λ -calculus with recursion. For this purpose explicit environments turn out to be useful, as we have different kinds of variables with different properties.

1 Introduction

This paper introduces the operational semantics theory [Pit97] that goes with the calculus of *linear* explicit substitutions [GdPR98], which is the basis of our linear functional programming implementation. The development is relatively straightforward, but whenever an operational semantics is used there are choices to be made and our commitment to explicit substitutions informs our choices.

To keep the paper short, we assume that the reader is familiar with Dual Intuitionistic and Linear Logic (henceforward DILL), as proposed by Barber and Plotkin [BP97], which we take as our starting point. Again there are choices as far as a *linear* lambda calculus is concerned, but here we exploit the fact that DILL has the simply typed lambda-calculus as its subcalculus, obtained when all the assumptions are intuitionistic.

To ‘linearize’ PCF we add to our chosen calculus DILL the typical PCF connectives, i.e. integers, booleans, conditionals and most importantly the fixpoint combinator, Y . The rules for Linear PCF are non-controversial and are described in detail, for instance by Braüner [Bra94]. Using our choice of explicit substitutions, explained and expounded on in [RdP97,GdPR98], we describe a linear environment calculus. For this linear environment calculus, we have both a call-by-name and a call-by-value operational semantics. We describe (non-surprising) translations between call-by-name PCF and its linear counterpart, and between call-by-value PCF and its linear counterpart. Then in the interesting part of this paper, using a traditional logical relations argument, we prove the call-by-name translation fully abstract (*i.e.*, preserves and reflects observational equivalence), and a similar sort of argument yields in the following section the same full abstraction for the call-by-value translation.

* Research supported under the EPSRC project no. GR/L28296, **x-SLAM: The Explicit Substitutions Linear Abstract Machine**.

It is surprising that an operational theory so analogous to the usual PCF one can be obtained for Linear PCF, especially considering the difficulties found in the literature with the ‘context lemma’. We would like to consider the naturality of the results here as evidence for the linear environment calculus we advocate here and elsewhere.

This work extends work by Hasegawa [Has99b]. Using logical relations for DILL, he proves that the embedding from multiplicative intuitionistic linear logic into Barber and Plotkin’s DILL satisfies full completeness. Also in an unpublished manuscript [Has99a] (which we became aware of only after submitting this paper) Hasegawa proves fullness of the Girard translation using logical relations. In both papers he does not consider any notion of observational equivalence, nor does he consider recursion or non-termination.

Crole [Cro98] investigates the relation between observational equivalence in linear logic and bisimulation. With tensor products he gives an example of two terms which are observationally equivalent but not bisimilar. This example shows that his definition of bisimilarity for tensor product, which considers both components separately, is problematic. Hasegawa’s categorically motivated logical relations for the tensor product, which use the elimination rule for the tensor product instead, should yield the desired coincidence of observational equality with logical relations.

Bierman [Bie97] takes a different view of the relationship between linear and intuitionistic logic. He uses linear logic as a metalanguage, and describes in [Bie97] translations from PCF into this metalanguage. These translations differ for call-by-name and call-by-value PCF. As a consequence he obtains different results about the relationship between observational equivalence in PCF and linear PCF. In contrast we view the linear λ -calculus as an intermediate language, and for us the translation is the same for both call-by-value and call-by-name PCF.

2 Linear PCF with environments

For completeness, we start with PCF [Plo77], which is the simply-typed λ -calculus with special expressions for natural numbers and booleans and unlimited recursion added. PCF has the following raw expressions:

$$t ::= x \mid \lambda x: A. t \mid tu \mid Yt \\ \mathbf{n} \mid \mathbf{succ } t \mid \mathbf{pred } t \mid \mathbf{true} \mid \mathbf{false} \mid \mathbf{iszero } t \mid \mathbf{if } t \mathbf{ then } t \mathbf{ else } t$$

The base types are ι (for natural numbers) and o (for booleans).

Linear PCF [Bra94] adds natural numbers, booleans and recursion to a linear λ -calculus rather than to the simply-typed λ -calculus. We use here Barber and Plotkin’s DILL (Dual Intuitionistic Linear Logic [BP97]) as the linear λ -calculus. This calculus has two kinds of variables: intuitionistic variables, which might be copied or discarded, and linear variables, which have to be used exactly once. This distinction is also reflected in the type system: A type $!A$ denotes a set of

terms which might be copied or discarded, and a type A denotes a set of terms which must be used exactly once. The idea is that an intuitionistic variable of type A is isomorphic to a linear variable of type $!A$. Natural numbers and boolean expressions are added in such a way that functions like successor or conditionals act on arguments of linear type rather than of $!$ -type whenever possible.

The recursion in linear PCF uses only the $!$ -types. The Y -combinator has the type $!(A \multimap A) \multimap A$, indicating that we may use the function we do recursion over possibly several times. This recursion does not take linearity into account; it is as powerful as the recursion in PCF.

The raw expressions of linear PCF are given by

$$\begin{aligned} t ::= & x \mid a \mid \lambda a: A. t \mid tt \mid !t \mid \text{let } t \text{ be } !x \text{ in } t \mid Yt \mid \\ & \mathbf{n} \mid \text{succ } t \mid \text{pred } t \mid \text{true} \mid \text{false} \mid \text{iszero } t \mid \text{if } t \text{ then } t \text{ else } t \\ A ::= & \iota \mid o \mid A \multimap A \mid !A \end{aligned}$$

where x, y, z etc. denote intuitionistic variables and a, b, c etc. denote linear variables.

We add to linear PCF environments and closures, where environments are lists of pairs of closed terms and variables indicating which term is to be substituted for which variable. A closure is the application of an environment to a term. The use of environments and closures is motivated by the unfoldings of recursions. Each unfolding of a recursion like $Y(\lambda x: A. t) \rightsquigarrow (\lambda x: A. t)(Y(\lambda x: A. t)) \rightsquigarrow t[Y(\lambda x: A. t)/x]$ produces a new substitution for the recursion variable x . We must not use call-by-value for this application, otherwise immediate non-termination would result. Hence we need to distinguish the recursion variable x from the other variables. As a result we have three different kinds of variables, with only two of them being part of the syntax of linear PCF.

In this paper we keep this distinction in the syntax by adding to linear PCF explicit bindings of terms for all three kinds of variables in the syntax. We call this calculus linear PCF with environments or LPCF_e for short. It has terms $t, s, u, \dots$, environments $f, g, \dots$, and closures $M, N, \dots$.

Definition 1. *The raw expressions of LPCF_e are given by*

$$\begin{aligned} t ::= & x \mid a \mid \lambda x: A. t \mid tu \mid !t \mid \text{let } t \text{ be } !x \text{ in } u \mid Yt \\ & \mathbf{n} \mid \text{succ } t \mid \text{true} \mid \text{false} \mid \text{iszero } t \mid \text{if } t \text{ then } t \text{ else } t \\ f ::= & \langle \rangle * t \mid \langle f, M/x \rangle \mid \langle f, M/a \rangle \mid \langle f, M/x_r \rangle \\ M ::= & f * t \\ A ::= & \iota \mid o \mid A \multimap B \mid !A \end{aligned}$$

We use $a, b, c, \dots$ for linear variables, $x, y, z, \dots$ for intuitionistic variables and x_r, y_r, z_r for recursion variables, which are special intuitionistic variables.

We identify terms up to α -equivalence, and use Barendregt's variable convention that bound and free variables are distinct. The typing rules for LPCF_e are given in Figure 1.

(i) For terms:

$$\begin{array}{c}
\Gamma, x: A, \Gamma' \vdash x: A \quad \Gamma \vdash a: A \vdash a: A \\
\\
\frac{\Gamma \mid \Delta, a: A \vdash t: B}{\Gamma \mid \Delta \vdash \lambda a: A. t: A \multimap B} \quad \frac{\Gamma \mid \Delta_1 \vdash t: A \multimap B \quad \Gamma \mid \Delta_2 \vdash u: A}{\Gamma \mid \Delta \vdash tu: B} \\
\\
\frac{\Gamma \mid \vdash t: A}{\Gamma \mid \vdash !t: !A} \quad \frac{\Gamma \mid \Delta_1 \vdash t: !A \quad \Gamma, x: A \mid \Delta_2 \vdash u: B}{\Gamma \mid \Delta \vdash \text{let } t \text{ be } !x \text{ in } u: B} \\
\\
\frac{}{\Gamma \mid \vdash \text{true}: o} \quad \frac{}{\Gamma \mid \vdash \text{false}: o} \\
\\
\frac{\Gamma \mid \Delta_1 \vdash t_1: o \quad \Gamma \mid \Delta_2 \vdash t_2: A \quad \Gamma \mid \Delta_2 \vdash t_3: A}{\Gamma \mid \Delta_1, \Delta_2 \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3: A} \\
\\
\frac{\Gamma \mid \Delta \vdash t: \iota}{\Gamma \mid \Delta \vdash \text{succ } t: \iota} \quad \frac{\Gamma \mid \Delta \vdash t: \iota}{\Gamma \mid \Delta \vdash \text{pred } t: \iota} \\
\\
\frac{}{\Gamma \mid \vdash \mathbf{n}: \iota} \quad \frac{\Gamma \mid \Delta \vdash t: \iota}{\Gamma \mid \Delta \vdash \text{iszero } t: o} \\
\\
\frac{\Gamma \mid \vdash t: !A \multimap A}{\Gamma \mid \vdash Yt: A}
\end{array}$$

In those rules with multiple premises, Δ_1, Δ_2 are disjoint and Δ is a permutation of Δ_1, Δ_2 .

(ii) For environments:

$$\begin{array}{c}
\frac{}{\vdash \vdash \langle \rangle: \vdash} \quad \frac{\vdash \vdash f: \Gamma \mid \Delta \quad \vdash \vdash g * t: A}{\vdash \vdash \langle f, g * t/x \rangle: (\Gamma, x: A \mid \Delta)} \\
\\
\frac{\vdash \vdash f: \Gamma \mid \Delta \quad \vdash \vdash g * t: A}{\vdash \vdash \langle f, g * t/x_r \rangle: (\Gamma, x_r: A \mid \Delta)} \quad \frac{\vdash \vdash f: \Gamma \mid \Delta \quad \vdash \vdash g * t: A}{\vdash \vdash \langle f, t/a \rangle: \Gamma \mid \Delta, a: A}
\end{array}$$

(iii) For Closures:

$$\frac{\vdash \vdash f: \Gamma \mid \Delta \quad \Gamma \mid \Delta \vdash t: A}{\vdash \vdash f * t: A}$$

Fig. 1. Typing rules for LPCF_e

Similarly, we extend PCF with environments and closures to obtain a calculus called PCF_e . The typing rules for environments and closures of PCF_e are

$$\frac{}{_ \vdash \langle \rangle : _} \quad \frac{_ \vdash f : \Gamma \quad _ \vdash g * t : A}{_ \vdash \langle f, g * t/x \rangle : (\Gamma, x : A)}$$

$$\frac{_ \vdash f : \Gamma \quad _ \vdash g * t : A}{_ \vdash \langle f, g * t/x_r \rangle : (\Gamma, x_r : A)} \quad \frac{_ \vdash f : \Gamma \quad \Gamma \vdash t : A}{_ \vdash f * t : A}$$

Next, we give the Girard-translation from PCF_e into LPCF_e .

Definition 2. (i) Define the Girard translation $_^{lin}$ from PCF_e into LPCF_e on terms as follows:

$$\begin{aligned} x^{lin} &= x \\ (\lambda x : A. t)^{lin} &= \lambda x : !A^{lin}. \text{let } !x \text{ be } x \text{ in } t^{lin} \\ (ts)^{lin} &= t^{lin} ! s^{lin} \\ (Yt)^{lin} &= Y t^{lin} \\ c(t_1, \dots, t_n)^{lin} &= c(t_1^{lin}, \dots, t_n^{lin}) \end{aligned}$$

where $c(t_1, \dots, t_n)$ is any of the special functions in PCF_e , and on environments and closures componentwise by translating an environment of PCF_e into an environment of LPCF_e with no linear variables. Define the Girard-translation of PCF_e -types into LPCF_e -types by

$$\begin{aligned} \iota^{lin} &= \iota \\ o^{lin} &= o \\ (A \rightarrow B)^{lin} &= !A^{lin} \multimap B^{lin} \end{aligned}$$

(ii) In the other direction, define a translation $(_)^{int}$ from LPCF_e to PCF_e on terms by

$$\begin{aligned} x^{int} &= x \\ a^{int} &= a \\ (\lambda a : A. t)^{int} &= \lambda a : A^{int}. t^{int} \\ (ts)^{int} &= t^{int} s^{int} \\ (!t)^{int} &= t^{int} \\ (\text{let } !x \text{ be } t \text{ in } s)^{int} &= s^{int}[t^{int}/x] \\ (Yt)^{int} &= Y t^{int} \\ (c(t_1, \dots, t_n))^{int} &= c(t_1^{int}, \dots, t_n^{int}) \end{aligned}$$

and on types by

$$\begin{aligned} \iota^{int} &= \iota \\ o^{int} &= o \\ (A \multimap B)^{int} &= A^{int} \rightarrow B^{int} \\ (!A)^{int} &= A^{int} \end{aligned}$$

and on environments and closures componentwise.

DILL has no λ -abstraction over intuitionistic variables defined as part of the syntax. However, the term $s = \lambda a: !A. \text{let } !x \text{ be } a \text{ in } t$ behaves exactly like such a λ -abstraction. Hence we will abbreviate s by $\lambda x: A. t$ in the sequel.

Both translations preserve typing:

Proposition 3. (i) Assume t is a PCF-term such that $\Gamma \vdash t: A$. Then we have $\Gamma^{lin} \mid _ \vdash t^{lin}: A^{lin}$.
(ii) Assume t is a linear PCF-term such that $\Gamma \mid \Delta \vdash t: A$. Then $\Gamma^{int}, \Delta^{int} \vdash t^{int}: A^{int}$.

3 Call-by-Name

In this section we show that the Girard translation $_^{lin}$ is fully abstract with respect to a call-by-names operational semantics.

This operational semantics specifies how closed terms are reduced to canonical terms, the so-called values. Values of type ι and o are natural numbers and booleans respectively, values of type $A \multimap B$ are closures $f * \lambda a: A. t$ (i.e., functions) and values of type $!A$ are closures $f * !t$ (i.e., terms which may be duplicated or discarded). Values are given inductively by

$$v ::= \text{true} \mid \text{false} \mid \mathbf{n} \mid f * \lambda a: A. t \mid f * !t$$

In this section we investigate a call-by-name operational semantics, where arguments of a function are reduced only when accessed. The clauses are listed in Figure 2.

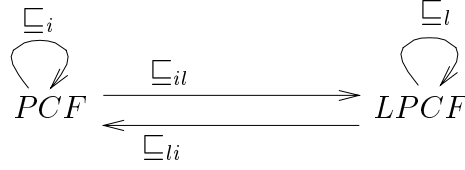
The notion of observational equivalence captures formally when two terms denote the same program: the observables are the values of ground type, and two terms are called *observationally equivalent* if when evaluated in any closing context they yield the same observable (i.e., value). In general it is very difficult to show that two terms are observationally equivalent according to the definition because one has to test them in all possible (arbitrarily complex) contexts. It turns out that for establishing observational equivalence it suffices to apply the two functions which we want to test for observational equivalence to all possible arguments. The fixed point combinator makes it necessary to define observational equivalence as the intersection of two approximation relations in the opposite direction. These approximation relations are the syntactic equivalent to the information order on domains, and the syntactic treatment of the fixed point combinator is analogous to its treatment as a supremum of finite approximations in this order. For an exposition of this well-known approach to showing observational equivalence see [Pit97].

The proof of the preservation of observational equivalence is structured as follows: we start by defining approximation relations

Let c be any of **n**, **true** or **false**. Let f_t be the environment which contains the bindings for all intuitionistic variables of f and for all free linear variables of t .

$$\begin{array}{c}
\frac{}{v \Downarrow_l v} \\
\\
\frac{}{f * c \Downarrow_l c} \\
\\
\frac{g * t \Downarrow_l v}{\langle f, g * t / x, t / \mathbf{x} \rangle * x \Downarrow_l v} \\
\\
\frac{g * t \Downarrow_l v}{\langle f, g * t / a, t / \mathbf{x} \rangle * a \Downarrow_l v} \\
\\
\frac{f_t * t \Downarrow_l g * \lambda a: A.t' \quad \langle g, f_s * s / a \rangle * t' \Downarrow_l v}{f * (ts) \Downarrow_l v} \\
\\
\frac{f_t * t \Downarrow_l g * !u \quad \langle f_s, g * u / x \rangle * s \Downarrow_l v}{f * \text{let } !x \text{ be } t \text{ in } s \Downarrow_l v} \\
\\
\frac{f * t \Downarrow_l g * \lambda a: !A.t' \quad \langle g, g * Y(\lambda a: !A.t') / x_r \rangle * t'[x_r / a] \Downarrow_l v}{f * (Yt) \Downarrow_l v} \\
\\
\frac{\langle g, g * (Y(\lambda a: !A.t)) / x_r \rangle * t[x_r / a] \Downarrow_l v}{\langle f, g * (Y(\lambda x: !A.t)) / x_r, t / \mathbf{x} \rangle * x_r \Downarrow_l v} \\
\\
\frac{f_{t_1} * t_1 \Downarrow_l \text{true} \quad f_{t_2} * t_2 \Downarrow_l v}{f * \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow_l v} \\
\\
\frac{f * t_1 \Downarrow_l \text{false} \quad f_{t_3} * t_3 \Downarrow_l v}{f * \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow_l v} \\
\\
\frac{f * t \Downarrow_l \mathbf{n}}{f * (\text{succ } t) \Downarrow_l \mathbf{n} + \mathbf{1}} \\
\\
\frac{f * t \Downarrow_l \mathbf{0}}{f * (\text{pred } t) \Downarrow_l \mathbf{0}} \\
\\
\frac{f * t \Downarrow_l \mathbf{n} + \mathbf{1}}{f * (\text{pred } t) \Downarrow_l \mathbf{n}} \\
\\
\frac{f * t \Downarrow_l \mathbf{0}}{f * (\text{iszero } t) \Downarrow_l \text{true}} \\
\\
\frac{f * t \Downarrow_l \mathbf{n} + \mathbf{1}}{f * (\text{iszero } t) \Downarrow_l \text{false}}
\end{array}$$

Fig. 2. Operational semantics for call-by-name LPCF_e



The key point is to show that for each approximation relation, a PCF-term approximates itself (or the translation into linear PCF, as appropriate). In the second step we use the relations $\sqsubseteq^{il}$ and $\sqsubseteq^{li}$ to show that the Girard translation maps equivalence classes of the observational equivalence in PCF to equivalence classes of the observational equivalence in LPCF.

The approximation relations are defined as logical relations, *i.e.*, we define a family of relations indexed by types in such a way that for ground types we specify the desired property, and we define two terms of function type to be related if they map related arguments to related results.

We start with the definition of the relation $\sqsubseteq^{il}$.

Definition 4. Define the approximation relation $\sqsubseteq_A^{il}$ between closures of PCF_e of type A and closures of $LPCF_e$ of type A^{lin} by induction over the structure of types as follows:

G : $M \sqsubseteq_G^{il} N$ if $M \Downarrow_l v$ implies $N \Downarrow_l v$.
 $A \rightarrow B$: $M \sqsubseteq_{A \rightarrow B}^{il} N$ if $M \Downarrow_l f * \lambda x: A. t$ implies $N \Downarrow_l g * \lambda x: A^{lin}. s$ and for all closures M' and N' such that $M' \sqsubseteq_A^{il} N'$ we have $\langle f, M'/x \rangle * t \sqsubseteq_B^{il} \langle g, N'/x \rangle * s$.

The key point is now to show that the Y -combinator can be approximated by certain λ -terms Y^k with

$$\begin{aligned} Y_A^0 &= \lambda x: A \rightarrow A. Y(\lambda x: A. x) \\ Y_A^{k+1} &= \lambda x: A \rightarrow A. x(Y_A^k x) \end{aligned}$$

In PCF_e , we call each application of the inference rules

$$\frac{f * t \Downarrow_l g * \lambda x: A. s \quad \langle g, g * Y(\lambda x: A. s)/x \rangle * s \Downarrow_l v}{f * (Yt) \Downarrow_l v}$$

$$\frac{\langle g, g * (Y(\lambda x: A. s))/x \rangle * s \Downarrow_l v}{\langle f, g * (Y(\lambda x: A. s))/x \rangle * x \Downarrow_l v}$$

an *unfolding* of the term Yt and $Y(\lambda x: A. s)$ respectively.

Note that we have two rules for the Y -combinator. The first rule reduces the term t to the function $\lambda x: A. s$ and does one unfolding of the recursion. Further unfoldings are done by the second rule. As an example, consider the PCF-term

$$M = (Y(\lambda x: \iota \rightarrow \iota. \lambda y: \iota. \text{if iszero } 0 \text{ then } 1 \text{ else } x(\text{pred } y)))1$$

If we denote by N the term `if iszero 0 then 1 else  $x(\text{pred } y)$` , then by the first unfolding rule, $M \Downarrow_l v$ if

$$\langle Y(\lambda x: \iota \rightarrow \iota. \lambda y: \iota. N)/x, 1/y \rangle * N \Downarrow_l v$$

If we denote by f the environment $\langle Y(\lambda x: \iota \rightarrow \iota. \lambda y: \iota. N)/x, 1/y \rangle$, then by the second unfolding rule, this is the case if

$$\langle Y(\lambda x: \iota \rightarrow \iota. \lambda y: \iota. N)/x, f * \text{pred } y/y \rangle * N \Downarrow_l v$$

Now it is easy to see that the second expression evaluates to 1.

If during reduction of a term a closure is unfolded at most k times, we can replace the fixed point combinator for this closure by the approximation Y^k . The following lemma gives the details.

Lemma 5. *Assume $M \Downarrow_l f * t$ in PCF_e and that there are k unfoldings of a closure $g * Yu$ with $g * u \Downarrow_l \lambda x: A. s$ during the evaluation of M . Then $M' \Downarrow_l f' * t$, where M' and f' result from M and f respectively by replacing every occurrence of $\langle g, g * (Y(\lambda x: A. s))/x \rangle$ by $g^i * (Y^{k-i}x)$ with $1 \leq i \leq k$, or with $\langle g, g * Y^k(\lambda y: A. s)/x \rangle$, where*

$$\begin{aligned} g^1 &\stackrel{\text{def}}{=} \langle g, g * \lambda y: A. s/x \rangle \\ g^{i+1} &\stackrel{\text{def}}{=} \langle g^i, g^i * x/x \rangle \end{aligned}$$

Proof. If $f * Yu \Downarrow_l v$, then the last step in the derivation must have been

$$\frac{f * u \Downarrow_l g * \lambda y: A. s \quad \langle g, g * Y(\lambda y: A. s/x) * s \Downarrow_l v \rangle}{f * (Yu) \Downarrow_l v}.$$

Now assume that there have been k occurrences of the unfolding rule for the closure $g * Y(\lambda y: A. s)$. Then show by an induction over the derivation that $\langle g, g * Y(\lambda y: A. s)/x \rangle * s \Downarrow_l f * t$ iff $\langle g, g^i * (Y^{k-i}x)/x \rangle * s \Downarrow_l f' * t$ for some i with $1 \leq i \leq k$.

The next lemma is the syntactic analogue expressed via the approximation relation of defining the semantics of Yt as the upper bound of all terms $Y^k t$.

Lemma 6. *If $f * Y^k t \sqsubseteq_A^{il} u$ for all k , then also $f * Yt \sqsubseteq_A^{il} u$.*

Proof. If $f * Yt$ diverges, the lemma is vacuously true. Hence assume there exists a k such that $f * Y^k t \Downarrow_l v$. Hence also $f * t \Downarrow_l g * \lambda x: A. u$ for some s . Now use Lemma 5 to show by an induction over the type structure that if $f' * t \sqsubseteq_A^{il} u$ for all k , where f' arises from f as described in Lemma 5, then also $f * t \sqsubseteq_A^{il} u$.

Now we can show the crucial proposition, namely the each PCF-term and its translation into LPCF are related.

Proposition 7. *For all closures M in PCF_e , we have $M \sqsubseteq_A^{il} M^{lin}$.*

Proof. Easy induction over the structure of M except for the case Yt . For this case, consider any two environments f and g such that $f * x_i \sqsubseteq_{A_i}^{il} g * x_i$. We have to show that $f * Yt \sqsubseteq_A^{il} g * Yt^{lin}$. If $f * Yt$ diverges, this is obvious. Otherwise, assume $f * t \Downarrow_l f' * \lambda x: A.t'$. Now show by an induction over k that $f' * Y^k(\lambda x: A.t') \sqsubseteq_A^{il} g * Yt^{lin}$. The case $k = 0$ is obvious because the LHS diverges. Now we turn to the induction step. We have that $f' * Y^{k+1}(\lambda x: A.t') \Downarrow_l v$ implies that $\langle f', \langle f', f' * \lambda x: A.t'/x \rangle * (Y^k x)/x \rangle * t' \Downarrow_l v$. Because x does not occur freely in Y^k , this means also that $\langle f', f' * (Y^k \lambda x: A.t')/x \rangle * t' \Downarrow_l v$. By induction hypothesis for k , $f' * (Y^k \lambda x: A.t') \sqsubseteq_A^{il} g * Yt^{lin}$. By induction hypothesis for t , we have $g * t^{lin} \Downarrow_l g' * \lambda x: A^{lin}.s'$, and $\langle f', u/x \rangle * t' \sqsubseteq_A^{il} \langle g', u'/x \rangle * s'$ if $u \sqsubseteq_A^{il} u'$. Hence if we specialise u to $f' * (Y^k \lambda x: A.t')$ and u' to $g * Yt^{lin}$, we obtain that $\langle f', f' * (Y^k \lambda x: A.t') \rangle * t' \sqsubseteq_A^{il} g * Yt^{lin}$. Because $\langle f', f' * (Y^k \lambda x: A.t')/x \rangle * t'$ and $f * Y^{k+1}t$ evaluate to the same value, we get $f * Y^{k+1}t \sqsubseteq_A^{il} g * Yt^{lin}$. Now Lemma 6 yields the claim.

If we define a relation R on PCF_e -closures by $M R N$ if $M \sqsubseteq_A^{il} N^{lin}$, then this proposition states that this relation is reflexive. The definition of $\sqsubseteq_A^{il}$ implies directly that R is transitive. Hence R is a pre-order.

Next we define the opposite approximation relation $\sqsubseteq_A^{li}$ between closures of LPCF_e and PCF_e in exactly the same way.

Definition 8. Define the approximation relation $\sqsubseteq_A^{li}$ between closures of LPCF_e of type A^{lin} and closures of PCF_e of type A by induction over the structure of types as follows:

$G: M \sqsubseteq_G^{li} N$ if $M \Downarrow_l v$ implies $N \Downarrow_l v$.
 $A \rightarrow B: M \sqsubseteq_{A \rightarrow B}^{li} N$ if $M \Downarrow_l f * \lambda x: A^{lin}.t$ implies $N \Downarrow_l g * \lambda x: A.s$ and for all closures M' and N' such that $M' \sqsubseteq_A^{li} N'$ we have $\langle f, M'/x \rangle * t \sqsubseteq_B^{li} \langle g, N'/x \rangle * s$.

In exactly the same way we obtain

Proposition 9. For all closures M in PCF_e , we have $M^{lin} \sqsubseteq_A^{li} M$.

We denote by $\cong^{il}$ the equivalence relation on closures of PCF_e given by $M \cong^{il} N$ if $M \sqsubseteq_A^{il} N^{lin}$ and $N^{lin} \sqsubseteq_A^{li} M$.

For the full abstraction of the translation, we use a characterisation of observational equivalence of PCF_e -closures as the intersection of $\sqsubseteq^i$ and its opposite relation, and similarly for LPCF_e . These approximation relations are defined by

Definition 10. (i) Let M and N closures of PCF_e . We define a family of relations $\sqsubseteq_A^i$ indexed by PCF-types by

- $M \sqsubseteq_G^i N$ if $M \Downarrow_l v$ implies $N \Downarrow_l v$;
- $M \sqsubseteq_{A \rightarrow B}^i N$ if $M \Downarrow_l f * \lambda x: A.t$ implies $N \Downarrow_l g * \lambda x: A.s$, and for all closures M' and N' such that $M' \sqsubseteq_A^i N'$, we have that $\langle f, M'/x \rangle * t \sqsubseteq_B^i \langle g, N'/x \rangle * s$.

(ii) Let M and N closures of LPCF_e . We define a family of relations $\sqsubseteq_A^l$ indexed by PCF-types by

- $M \sqsubseteq_G^l N$ if $M \Downarrow_l v$ implies $N \Downarrow_l v$;
- $M \sqsubseteq_{A \multimap B}^l N$ if $M \Downarrow_l f * \lambda a: A. t$ implies $N \Downarrow_l g * \lambda a: A. s$, and for all closures M' and N' such that $M' \sqsubseteq_A^l N'$, we have that $\langle f, M'/a \rangle * t \sqsubseteq_B^l \langle g, N'/a \rangle * s$;
- $M \sqsubseteq_{!A}^l N$ if $M \Downarrow_l f * !t$ implies $N \Downarrow_l g * !s$, and $f * t \sqsubseteq_A^l g * s$.

Similarly to $\cong^{il}$, one defines equivalence relations $\cong^i$ ($\cong^l$) as the intersection of $\sqsubseteq^i$ ($\sqsubseteq^l$) and its opposite relation. Because the languages we are considering are purely functional we have the following Theorem

Theorem 11. *The relation $\cong^i$ restricted to PCF-terms coincides with observational equivalence on PCF, and $\cong^l$ restricted to LPCF-terms coincides with observational equivalence on LPCF.*

Proof. Given two PCF-terms t and s which are related by $\cong^i$, one shows by an induction over the structure of contexts $C[_]$ that $C[t]$ and $C[s]$ are related by $\cong^i$. Proposition 9 is crucial for establishing the application case.

Now assume two PCF-terms are observationally equivalent. One shows by an induction over the type structure that they are also equivalent modulo $\cong^i$.

This argument works in the same way for linear PCF.

Crole [Cro98] uses bisimulations to reason about observational equivalence for DILL with a non-terminating term added. Because he defines bisimilarity for the tensor product $A \otimes B$ simply in terms of bisimilarity for A and for B , bisimilarity and observational equivalence do not coincide in his case. As we do not consider tensor products here, this problem does not arise. If we added tensor products, Hasegawa's approach to logical relations [Has99b], which essentially defines the logical relation for the tensor product via its elimination rule, should work.

One shows with a similar method to Proposition 9

Proposition 12. *For all closures of the linear environment calculus of type A^{lin} , one has $M \cong^l (M^{int})^{lin}$.*

Now we can show the main theorem.

Theorem 13. *The translation $_^{lin}$ of call-by-name PCF into linear call-by-name PCF is fully abstract.*

Proof. First, assume that M and N are observationally equivalent closures in the environment calculus. Hence $M \cong^i N$, and by proposition 7, we have also $M \cong^{il} M^{lin}$ and $N \cong^{il} N^{lin}$. Using Proposition 12, one shows now by induction over the PCF-types that $M^{lin} \cong^l N^{lin}$. Hence M^{lin} and N^{lin} are observationally equivalent closures in the linear environment calculus.

For the other direction one shows by an induction over PCF-types that $M^{lin} \cong^l N^{lin}$ implies $M \cong^i N$.

4 Call-by-Value

In this section we show that the results for the call-by-name calculus hold also for a call-by-value calculus.

In the call-by-value operational semantics we evaluate arguments of functions before evaluating the body. Hence environments store only values. Values and environments for a call-by-value LPCF_e are now inductively defined as follows:

$$\begin{aligned} f_v &::= \langle \rangle \mid \langle f_v, v/x \rangle \mid \langle f_v, f_v * (Yt)/x_r \rangle \\ v &::= \mathbf{true} \mid \mathbf{false} \mid \mathbf{n} \mid f_v * \lambda x: A. t \mid !v \end{aligned}$$

The terms are the terms of call-by-name LPCF_e . Because in the sequel only environments which are values are considered we drop the subscript in f_v . We define the call-by-value PCF_e -calculus in the same way. The operational semantics is given by the clauses in Figure 3.

In a call-by-value calculus the distinction between recursive variables and intuitionistic variables is important: if we treated recursive variables like intuitionistic ones and replaced *e.g.*, the first unfolding rule by

$$\frac{f * t \Downarrow_v g * \lambda a: !A. t' \quad g * Y(\lambda a: !A. t') \Downarrow_v v' \quad \langle g, v'/x_r \rangle * t'[x_r/a] \Downarrow_v v}{f * (Yt) \Downarrow_v v}$$

each closure $f * Y(\lambda a: !A. t)$ would always diverge: by this rule, one condition for $f * Y(\lambda a: !A. t) \Downarrow_v v$ to hold is exactly $f * Y(\lambda a: !A. t) \Downarrow_v v$. Hence we treat recursive variables as in the call-by-name calculus.

The full abstraction result is still true for the call-by-value calculi. The same argument as in the previous section works. Note that in particular the approximants for the fixed point are defined using recursive variables. In this way the treatment of the fixed point combinator in the call-by-name calculus carries over directly to the call-by-value case.

Theorem 14. *The translation $_^{lin}$ of call-by-value PCF into linear call-by-value PCF is fully abstract.*

Proof. Similar to proof of Theorem 13.

5 Conclusions

This paper is part of a collection, which describe the results of the xSLAM project, currently being undertaken in the School of Computer Science, University of Birmingham. This project's main goal is to improve the efficiency of functional programming, by exploring the notion of linearity, as a logical tool. The general structure of the project is as follows:

Let c be any of **n**, **true** or **false**. Let f_t be the environment which contains the bindings for all intuitionistic variables of f and for all free linear variables of t .

$$\begin{array}{c}
\frac{}{v \Downarrow_v v} \\
\\
\frac{}{f * c \Downarrow_l c} \\
\\
\frac{}{\langle f, v/x, \mathbf{t}/\mathbf{x} \rangle * x \Downarrow_v v} \\
\\
\frac{}{\langle f, v/a, \mathbf{t}/\mathbf{x} \rangle * x \Downarrow_v v} \\
\\
\frac{f_t * t \Downarrow_v g * \lambda a: A.t' \quad f_s * s \Downarrow_v v' \quad \langle g, v'/a \rangle * t' \Downarrow_v v}{f * (ts) \Downarrow_v v} \\
\\
\frac{f_t * t \Downarrow_v !v \quad \langle f_s, v/x \rangle * s \Downarrow_v v'}{f * \text{let } !x \text{ be } t \text{ in } s \Downarrow_v v'} \\
\\
\frac{f * t \Downarrow_v v}{f * !t \Downarrow_v !v} \\
\\
\frac{f * t \Downarrow_v g * \lambda a: !A.t' \quad \langle g, g * Y(\lambda a: !A.t')/x_r \rangle * t'[x_r/a] \Downarrow_v v}{f * (Yt) \Downarrow_v v} \\
\\
\frac{\langle g, g * (Y(\lambda a: !A.t))/x_r \rangle * t[x_r/a] \Downarrow_v v}{\langle f, g * (Y(\lambda x: !A.t))/x_r, \mathbf{t}/\mathbf{x} \rangle * x_r \Downarrow_v v} \\
\\
\frac{f_{t_1} * t_1 \Downarrow_v \mathbf{true} \quad f_{t_2} * t_2 \Downarrow_v v}{f * \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow_v v} \\
\\
\frac{f * t_1 \Downarrow_v \mathbf{false} \quad f_{t_3} * t_3 \Downarrow_v v}{f * \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow_v v} \\
\\
\frac{f * t \Downarrow_v \mathbf{n}}{f * (\text{succ } t) \Downarrow_v \mathbf{n} + \mathbf{1}} \\
\\
\frac{f * t \Downarrow_v \mathbf{0}}{f * (\text{pred } t) \Downarrow_v \mathbf{0}} \\
\\
\frac{f * t \Downarrow_v \mathbf{n} + \mathbf{1}}{f * (\text{pred } t) \Downarrow_v \mathbf{n}} \\
\\
\frac{f * t \Downarrow_v \mathbf{0}}{f * (\text{iszero } t) \Downarrow_v \mathbf{true}} \\
\\
\frac{f * t \Downarrow_v \mathbf{n} + \mathbf{1}}{f * (\text{iszero } t) \Downarrow_v \mathbf{false}}
\end{array}$$

Fig. 3. Operational semantics for call-by-value LPCF_e

- (1) Categorical Semantics for (Linear) Explicit Substitutions
- (2) Calculus of Linear Explicit Substitutions
- (3) Operational Semantics
- (4) Abstract Reduction Machine xLIN
- (5) Applications

We have described our proposed calculus of Linear Explicit Substitutions in [GdPR98] and our (categorical) models, which work as a guiding line in [GdPR99]. Here we discuss the operational theory. We show that the transition from PCF to linear PCF preserves and reflects observational equivalence. This seems to justify to use linear PCF to optimise the execution of PCF-programs. In fact, we would like to have a stronger result. To exploit linearity properly we require a translation of PCF into linear PCF which uses as many linear variables as possible. If we restrict ourselves to refinements of the Girard translation we expect the results of this paper to hold in this case as well.

Clearly the fixpoint combinator, as generally construed in the literature [Bra94] and as we use it here is not *linearised* as much as one would like it to be. But it is an important feature of Girard's linear logic that we can keep the (intuitionistic) traditional results that were obtained before. This note follows this line and obtains traditional full abstraction results for linear lambda-calculi using conventional (logical relations) arguments. We hope for an improved understanding of linear recursion in the near future.

References

- [Bie97] G.M. Bierman. Observations on a linear PCF. Technical Report 412, University of Cambridge Computer Laboratory, 1997.
- [BP97] A. Barber and G. Plotkin. Dual intuitionistic linear logic. Technical report, LFCS, University of Edinburgh, 1997.
- [Bra94] T. Braüner. The Girard translation extended with recursion. In *Proc. of CSL'94*, volume 933 of *Lecture Notes in Computer Science*. Springer Verlag, 1994.
- [Cro98] R.L. Crole. Soundness and incompleteness of bisimilarity for contextual equivalences in linear theories. Submitted, 1998.
- [GdPR98] N. Ghani, V. de Paiva, and E. Ritter. Linear explicit substitutions. In *Proc. of Westapp'98*, 1998. Full version submitted for publication.
- [GdPR99] Neil Ghani, Valeria de Paiva, and Eike Ritter. Categorical models of explicit substitutions. In *Proc. of FoSSaCS'99*, Lecture Notes in Computer Science, 1999.
- [Has99a] M. Hasegawa. Girard translation is full. Submitted, 1999.
- [Has99b] M. Hasegawa. Logical predicates for intuitionistic linear type theories. In *Proceedings, Fourth International Conference on Typed Lambda Calculi and Applications (TLCA'99)*, Lecture Notes in Computer Science. Springer-Verlag, 1999.
- [Pit97] A.M. Pitts. Operationally based theory of program equivalence. In P. Dybjer and A.M. Pitts, editors, *Semantics and Logics of Computation*. Cambridge University Press, 1997.

- [Plo77] G. Plotkin. LCF considered as a programming language. *Theoretical Computer Science*, 5:223–255, 1977.
- [RdP97] E. Ritter and V. de Paiva. On explicit substitution and names (extended abstract). In *Proc. of ICALP'97*, LNCS 1256, pages 248–258, 1997.